# Report 1: Implementation of Gradient Descent

Le Hoai Nam - 2540079

May 2026

## 1 Introduction

The objective of this task is to implement the Gradient Descent algorithm from scratch to find the minimum of the function $f(x) = x^2$. Gradient descent is a first-order optimization algorithm that iteratively moves toward the minimum of a function by following the direction of the negative gradient.

## 2 Implementation

The algorithm is implemented by defining the function and its first-order derivative. In each iteration, the value of $x$ is updated using the rule:

$$x_{new} = x_{old} - r \cdot f'(x_{old}) \tag{1}$$

where $r$ is the learning rate.

The algorithm is implemented following these logical steps:

1. **Initialization**: Define the starting point $x = 10$, learning rate $lr$, and the stopping condition $threshold = 0.01$.
2. **Gradient Calculation**: Compute the first-order derivative $df(x) = 2x$.
3. **Update Rule**: In each iteration, update $x$ using the formula:

$$x_{new} = x_{old} - lr \times df(x_{old})$$

4. **Monitoring**: Print the current count, the new value of $x$, the gradient $df(x)$, and the function value $f(x)$ for each step.
5. **Termination**: Repeat the update until the stopping condition $|f(x)| \leq threshold$ is met.

## 3 Analysis of Learning Rate ($r$)

The choice of the learning rate $r$ is critical for the performance and convergence of the algorithm:

- **Small Learning Rate ($r = 0.01$):** The algorithm converges very slowly. While it is stable, it requires a high number of iterations to reach the threshold.
- **Moderate Learning Rate ($r = 0.1$):** As seen in the results, the algorithm converges efficiently in 21 steps. This value provides a good balance between speed and stability.

- **Large Learning Rate ($r = 0.9$):** The algorithm may oscillate. If $r$ is close to 1.0 for this specific function, the update $x - 0.9(2x)$ causes $x$ to jump between positive and negative values.
- **Divergent Learning Rate ($r > 1.0$):** If $r = 1.1$, the step size is larger than the distance to the minimum. The value of $x$ will grow larger in each iteration, failing to converge.

## 4 Experimental Results ($r = 0.1$)

With an initial $x = 10$ and a threshold of 0.01 for $f(x)$, the algorithm produced the following iterative steps:

| **Step** | $x$ | $f'(x)$ | $f(x)$ |
|---|---|---|---|
| 1 | 8.0000 | 16.0000 | 64.0000 |
| 2 | 6.4000 | 12.8000 | 40.9600 |
| 3 | 5.1200 | 10.2400 | 26.2144 |
| ... | ... | ... | ... |
| 10 | 1.0737 | 2.1475 | 1.1529 |
| 20 | 0.1153 | 0.2306 | 0.0133 |
| 21 | 0.0922 | 0.1845 | 0.0085 |

## 5 Conclusion

The implementation successfully finds the minimum of $f(x) = x^2$. The experiment demonstrates that while Gradient Descent is powerful, it is highly sensitive to the hyperparameter $r$. Proper tuning is required to ensure both convergence and computational efficiency.